

ALGORITHM ANALYSIS REPORT

배열 합산 알고리즘 시간복잡도 분석

반복문 · 재귀 · 분할 정복 비교 연구

과목: 자료구조 및 알고리즘

분석 대상: E1ArraySum.java

작성일: 2026년 3월 26일

1. 개요

본 리포트는 정수 배열의 모든 원소의 합을 구하는 세 가지 알고리즘—반복문(Loop), 재귀(Recursion), 분할 정복(Divide & Conquer)—을 구현하고, 각각의 시간복잡도와 공간복잡도를 이론적으로 분석한다. 세 알고리즘 모두 동일한 결과를 반환하지만, 내부 동작 방식과 자원 사용량에 차이가 있다.

2. 알고리즘 개요 비교

메소드	방식	시간복잡도	공간복잡도
sumWithLoop	반복문	$O(n)$	$O(1)$
sumWithRecursion	재귀	$O(n)$	$O(n)$
sumWithDivideAndConquer	분할 정복	$O(n)$	$O(\log n)$

3. 알고리즘별 상세 분석

3-1. 반복문 (sumWithLoop)

```
public static int sumWithLoop(int[] arr) {  
    int sum = 0;  
    for (int i = 0; i < arr.length; i++) {  
        sum += arr[i];    // arr.length번 실행  
    }  
    return sum;  
}
```

동작 원리

배열 인덱스 0부터 $n-1$ 까지 순차적으로 순회하며 누적합을 계산한다. 반복문은 정확히 n 번 실행된다.

시간복잡도 분석

배열의 크기를 n 이라 할 때, 반복문은 $i = 0, 1, 2, \dots, n-1$ 으로 정확히 n 번 실행된다.

$$T(n) = n \rightarrow O(n)$$

공간복잡도 분석

추가로 사용하는 변수는 `sum`, `i` 두 개뿐이며, `n`에 무관하게 고정된 메모리를 사용한다.

$$S(n) = O(1) \text{ (최적)}$$

3-2. 재귀 (sumWithRecursion)

```
public static int sumWithRecursion(int[] arr, int index) {  
    if (index == arr.length) return 0;           // 기저 조건  
    return arr[index] + sumWithRecursion(arr, index + 1); //  
    재귀 호출  
}
```

동작 원리

현재 인덱스의 원소 값에 `index+1`부터의 합을 재귀적으로 더한다. `index`가 `arr.length`에 도달하면 0을 반환하여 재귀를 종료한다.

시간복잡도 분석

점화식으로 표현하면 다음과 같다.

$$T(n) = T(n-1) + O(1), T(0) = O(1)$$

이를 전개하면:

$$T(n) = T(n-1) + 1 = T(n-2) + 2 = \dots = T(0) + n = O(n)$$

공간복잡도 분석

재귀 호출은 호출 스택(Call Stack)에 프레임을 쌓는다. `index = 0`부터 `n`까지 총 **`n+1`**개의 스택 프레임이 동시에 메모리에 존재한다.

$$S(n) = O(n) \text{ (스택 오버플로우 주의)}$$

△ `n`이 매우 클 경우(Java 기본 스택 약 500~1000 depth) `StackOverflowError`가 발생할 수 있다.

3-3. 분할 정복 (sumWithDivideAndConquer)

```
public static int sumWithDivideAndConquer(int[] arr, int start,  
int end) {  
    if (start == end) return arr[start];           // 기저 조건
```

```

int mid = (start + end) / 2;
int leftSum = sumWithDivideAndConquer(arr, start,
mid);    // 왼쪽
int rightSum = sumWithDivideAndConquer(arr, mid + 1,
end);    // 오른쪽
return leftSum + rightSum;
}

```

동작 원리

배열을 절반씩 분할(Divide)하여 각 부분의 합을 재귀적으로 계산하고, 두 결과를 합산(Conquer)한다. 분할이 계속되어 크기가 1이 되면 해당 원소를 반환한다.

재귀 트리 (예: {1, 2, 4, 8, 16})

```

      [0, 4] → 31
      /
    /   \
  /       \
 /         \
[0, 1] → 3  [2] → 4  [3] → 8  [4] → 16
/   \
[0] → 1  [1] → 2

```

시간복잡도 분석

마스터 정리(Master Theorem)를 적용한다.

$$T(n) = 2T(n/2) + O(1)$$

$a = 2, b = 2, f(n) = O(1) \rightarrow n^{\log_2 2} = n^1 = n$ 이므로 Case 1에 해당:

$$T(n) = O(n)$$

공간복잡도 분석

재귀 트리의 깊이는 배열을 절반씩 나누므로 $\log_2 n$ 이다. 동시에 스택에 존재하는 최대 프레임 수는 트리 깊이에 비례한다.

$$S(n) = O(\log n)$$

4. 종합 비교 분석

항목	반복문	재귀	분할 정복
시간복잡도	$O(n)$	$O(n)$	$O(n)$
공간복잡도	$O(1)$	$O(n)$	$O(\log n)$

스택 사용	없음	n+1 프레임	log n 프레임
코드 가독성	높음	높음	보통
대용량 데이터	안전	위험 (SO 오류)	비교적 안전
실무 활용	★★★	★★☆	★★☆

5. 결론

- 세 알고리즘 모두 **시간복잡도 $O(n)$** 으로 동일하다. 배열의 모든 원소를 최소 1회씩 접근해야 하므로 $O(n)$ 미만은 불가능하다.
- **공간복잡도**는 반복문($O(1)$) < 분할 정복($O(\log n)$) < 재귀($O(n)$) 순으로 효율적이다.
- 실무에서는 **반복문**이 가장 효율적이고 안전하다.
- 재귀는 코드가 직관적이나 대규모 배열에서 StackOverflowError 위험이 있다.
- 분할 정복은 이 문제에서 이점이 크지 않으나, 병렬 처리(Parallel Sum) 확장 시 유리한 구조이다.